

PatchWise A2A_CLI: Autonomous Agent2Agent Patch Review for Linux Kernel Workflows

Ajay Nandam (Project Owner), A2A_CLI Engineering Workspace

Abstract: Linux kernel patch review is often iterative, asynchronous, and manually intensive when using LLM-assisted workflows. Engineers spend significant time supervising agents, replaying context, and validating that reviewer decisions are consistent with evidence. This paper presents PatchWise A2A_CLI, a local Agent2Agent orchestration system that structures builder-reviewer collaboration into auditable rounds with explicit findings, policy-gated LGTM decisions, lore-thread ingestion, and automatic respin generation. The system separates implementation and review concerns, persists full artifacts per round, and enables resumable workflows from local patch files or lore links. Initial deployment in real patch-review sessions shows practical reductions in operator babysitting and improved traceability of technical decisions.

Keywords: Linux kernel, patch review, autonomous agents, Agent2Agent, lore.kernel.org, respin automation, evidence-based review.

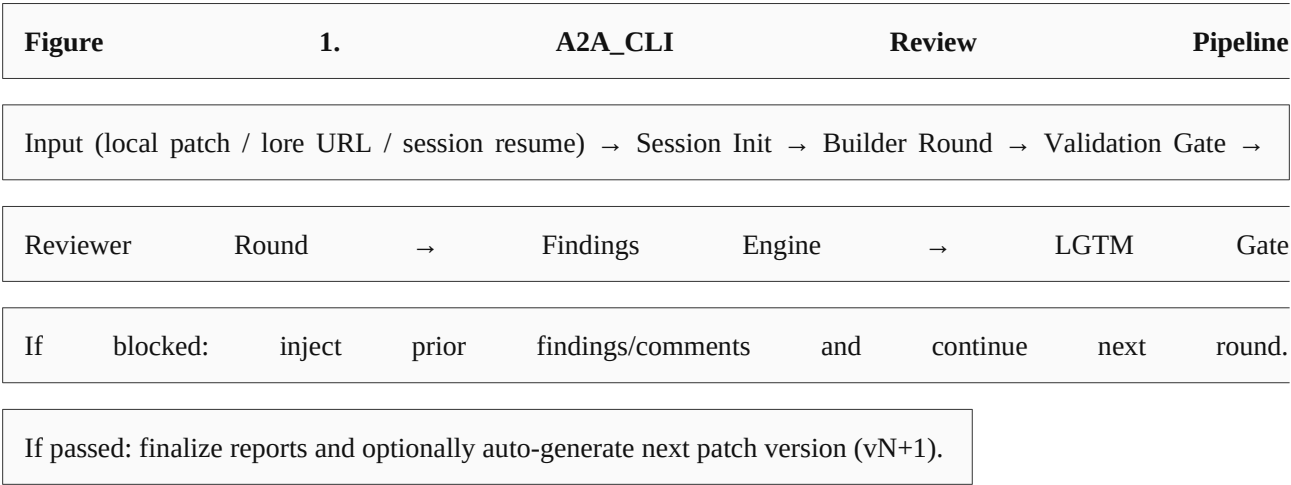
1. Problem Statement

Traditional AI-assisted review loops frequently fail in two ways: (1) weak role separation where the reviewer becomes non-adversarial, and (2) incomplete context continuity across rounds, leading to repeated findings and manual restarts. In kernel workflows, these failures are expensive because reviewers must verify patch integrity, logical split quality, and subsystem-specific correctness under strict maintainer expectations.

2. Proposed System

PatchWise A2A_CLI implements a deterministic state machine for autonomous rounds with strict evidence contracts:

- **Chanakya (Builder):** edits and fixes, must provide line-level evidence and residual risks.
- **Aryabhatta (Reviewer):** adversarially validates, emits structured findings with severity, location, evidence, and required action.
- **Orchestrator:** enforces round transitions, validation gates, and LGTM blocking policies.



3. Policy and Guardrails

Guardrail	Enforcement
Open findings safety	LGTM is blocked when open findings > 0.
Reviewer consistency	LGTM blocked if reviewer reasoning still contains unresolved concern/risk.
Prior-thread continuity	Lore comments ingested and mapped via source identifiers; unresolved items carried forward.
Concurrency safety	Worktree lock prevents concurrent loops from corrupting shared prepared trees.

4. Implementation Snapshot

The system is implemented as a local CLI with session state under `.a2a/`. It supports autonomous loop execution, report generation (JSON/Markdown/HTML), lore fetch integration, auto-respin workflows, and optional email bridge controls for remote operation.

```
Representative command: a2a loop --task <name> --lore-url "https://lore.kernel.org/all/<msgid>/" --max-rounds 3
```

5. Validation Status

Current project test suite in the local workspace passes successfully (168 passed, 1 skipped). Added validations include post-respin artifact coherence checks, cover-letter changelog quality checks, and worktree locking behavior tests.

6. Contributions

- Practical Agent2Agent orchestration for kernel patch review with strict role contracts.
- Evidence-driven findings lifecycle and deterministic LGTM gating.
- Lore-aware review continuity, including prior-comment ingestion and response scaffolding.
- Respin automation with patchset consistency checks.
- Operator-friendly execution via CLI loop, resume, report generation, and launcher script.

7. Limitations and Next Steps

Current results are based on engineering deployment and integration validation rather than a controlled benchmark across multiple subsystems. Next work includes formal metrics (rounds-to-LGTM, false-positive rate, review latency), broader subsystem datasets, and SPARQ-template-specific formatting alignment.

8. Reproducibility

All artifacts are generated locally per session in the project repository. A typical run emits round logs, findings, diffs, summary payloads, and an HTML session report under `.a2a/reports/<session-id>/`, enabling audit and replay.

Prepared for: SPARQ paper submission draft v1 (editable DOCX + PDF export).